

Sicurezza

Tracce Homework

a.a. 25/26

L'homework è un lavoro di gruppo: minimo 2, massimo 3 studenti.

Traccia 1 - Stack Buffer Overflow

Realizzare un attacco di stack buffer overflow [1] che permetta di aprire una shell su di un sistema target. A tale scopo si richiede sia di realizzare un programma vulnerabile ad un attacco di buffer overflow, sia di progettare e implementare la sequenza di byte che deve essere iniettata nel buffer per realizzare l'attacco. La shell deve essere eseguita con i privilegi del programma vulnerabile che viene sfruttato per il buffer overflow.

(Opzionale) migliorare l'attacco provando a far guadagnare alla shell maggiori privilegi rispetto a quelli del programma di cui si è sfruttata la vulnerabilità.

Suggerimenti:

- Installare una versione di SO vulnerabile ad attacchi overflow ovvero disabilitare tutte le impostazioni del sistema operativo che permettono di prevenire attacchi di buffer overflow.
- Considerare codice oggetto a 32bit

[1] Buffer Overflow in an API Call, <https://capec.mitre.org/data/definitions/8.html>

Traccia 2 - SQL Injection

Realizzare un attacco di SQL injection [1] di tipo inband, basato su input dell'utente e che utilizzi una o più delle seguenti modalità: Tautologia, commento di fine riga; query piggybacked.

Mediante l'injection di opportuni comandi mostrare che è possibile violare le tre proprietà CIA del database. Nello specifico, supponendo che il database è accessibile mediante un applicazione web vulnerabile e che l'attaccante non conosca la struttura del database a priori:

- Raccogliere informazioni sulla struttura del DB o sulla sua configurazione
- Esfiltrare dati contenuti in una o più tabelle del database
- Modificare il contenuto del database, ad esempio modifica i valori di un campo all'interno di una tabella
- Cancellare una o più tabelle, o il loro contenuto

Proporre ed implementare misure di difesa a livello di Database e a livello di applicazione web e mostrare la loro efficacia

Suggerimenti:

- Supponete che il database sia accessibile mediante una applicazione web, ad esempio una pagina Php che genera query al DB
- Supponete che l'applicazione web sia vulnerabile (ad esempio non usi query parametrizzate o controlli sull'input)
- Il database server ed il web server possono essere emulati mediante docker container (o virtual machines)
- Scegliere versioni del software (OS, DB, Php, etc...) che siano vulnerabili ad attacchi SQLi o disabilitare eventuali meccanismi di protezione.

[1] SQL Injection, <https://capec.mitre.org/data/definitions/66.html>

Traccia 3 - Cross-Site Scripting

Realizzare un attacco di Cross Site Scripting, ad esempio XSS Through HTTP Headers [1] o Stored XSS [2].

Proporre ed implementare misure di difesa, mostrando la loro efficacia

Suggerimenti:

- Utilizzare un docker container per emulare un server web con relativa applicazione web vulnerabile (da voi realizzata)
- Connettersi al server web mediante un browser vulnerabile

[1] XSS Through HTTP Headers, <https://capec.mitre.org/data/definitions/86.html>

[2] Stored XSS, <https://capec.mitre.org/data/definitions/592.html>

Fonti Utili

1. Common Attack Pattern Enumerations and Classifications (CAPEC), <https://capec.mitre.org/index.html>
2. Common Weakness Enumeration (CWE), <https://cwe.mitre.org/index.html>
3. MITRE ATT&CK® Matrix for Enterprise, <https://attack.mitre.org/matrices/enterprise/>
4. Metasploitable3 <https://github.com/rapid7/metasploitable3>
5. Metasploit <https://github.com/rapid7/metasploit-framework>